# *MOTORMINDER*

Project Outline

*Ethan Hess, Ethan Kidd, Preston Little, Ryan Carbine*

*The Backlog Blackhole*

## Table of Contents



# Project Description

This project proposes the development of a car maintenance tracking and recommendation application designed to help vehicle owners better understand, manage, and maintain their vehicles' health. A lot of drivers have a hard time keeping track of routine maintenance, like oil changes, tire rotations, inspections, etc. This can lead to unnecessary repairs, wasted money, and reduced vehicle lifespan. This application aims to consolidate vehicle information to give the user valuable maintenance recommendations based on vehicle condition data.

# Software Requirement Specification

## Functional Requirements

FR1: Mileage Input

- FR1.1: The system shall allow users to input current vehicle milage as a positive integer.
- FR1.2: The system shall reject non-numeric or zero/negative milage input and display an error message within 1 second.
- FR1.3: The system shall reject mileage entries lower than the most recently recorded mileage and display an error message indicating the minimum accepted value.

FR2: Maintenance Item Management

- FR2.1: The system shall store at least 10 standard maintenance items with their recommended service intervals.
- FR2.2: The system shall allow users to record the mileage at which each maintenance item was last performed, accepting values between 0 and 999,999 miles.
- FR2.3: The system shall calculate the next due mileage for each maintenance item based on last service mileage + service interval, displayed as a whole number in miles.

FR3: Maintenance Recommendations

- FR3.1: The system shall display all maintenance items that are overdue (current mileage >= next due mileage).
- FR3.2: The system shall display maintenance items due within 1,000 miles of current mileage.
- FR3.3: The system shall categorize each maintenance item as "Overdue" (current mileage >= next due mileage), "Due Soon" (next due mileage is within 1,000 miles of current mileage), or "OK" (next due mileage is more than 1,000 miles away).

FR4: Service History

- FR4.1: The system shall allow users to mark a maintenance item as completed, recording the completion date (MM/DD/YYYY format) and mileage at time of completion.
- FR4.2: The system shall retain a complete history of all completed maintenance entries, each including item name, completion date, and mileage, with no automatic deletion.

FR5: Multiple Vehicles

- FR5.1: The system shall allow users to add up to *4* vehicles to their account.
- FR5.2: The system shall prevent users from adding more than 4 vehicles and display an error message when the limit is reached.

- FR5.3: The system shall require each vehicle to have a unique nickname or license plate identifier within a user's account, between 1 and 30 characters.
- FR5.4: The system shall display a list of all vehicles showing vehicle name and an overall status of "Overdue", "Due Soon", or "OK" based on the highest-priority maintenance item for that vehicle.
- FR5.5: The system shall allow users to select a specific vehicle and display its detailed maintenance recommendations within 2 seconds.

## Non-Functional Requirements

NFR1: Performance

- NFR1.1: The system shall calculate and display maintenance recommendations within 2 seconds of milage input.
- NFR1.2: The system shall support at least 100 concurrent users without response time exceeding 2 seconds for any individual request.

NFR2: Usability

- NFR2.1: The system shall be accessible via standard web browsers (Chrome, Firefox, Safari, Edge) without requiring additional plugins or extensions.
- NFR2.2: Users shall be able to input mileage and view recommendations in 3 clicks or less from the home page.
- NFR2.3: The system shall display error messages that clearly indicate the problem and the corrective action required.

NFR3: Reliability

- NFR3.1: The system shall maintain 99% uptime during business hours (8AM - 8PM local time), measured on a monthly basis.
- NFR3.2: User data shall be persisted and survive system restarts without loss, verified by successful data retrieval following any unplanned restart.

NFR4: Security

- NFR4.1: The system shall require authenticated login before granting access to any vehicle or maintenance data.
- NFR4.2: Passwords shall be stored using a cryptographic hashing algorithm with a minimum of 256-bit security (e.g., bcrypt or SHA-256).

NFR5: Maintainability

- NFR5.1: The system shall use a modular architecture such that any single component can be updated or replaced without modifying more than one other module.

- NFR5.2: Code shall include inline comments for all complex business logic sections, with a minimum of one comment per function exceeding 15 lines.

# Design

## Design Patterns

MotorMinder is a Python-based CLI application for vehicle maintenance tracking. The project is architected to ensure scalability, maintainability, and clear separation of concerns. This documentation summarizes the design patterns incorporated into the project.

### Model-View-Controller (MVC)

- **Model:** Handles data logic and persistence (`models.py`, `data_handler.py`).
- **View:** Manages CLI formatting and user output (`view.py`).
- **Controller:** Bridges user actions and model updates (`controller.py`).
- **CLI:** Entry point for user interaction (`cli.py`).

**Why MVC:** MotorMinder's core challenge is keeping the maintenance recommendation logic independent from how it is displayed. Without MVC, a change to the terminal output format would require digging through the same file that calculates service intervals, making both harder to test and modify. By placing CLI interaction in `cli.py`, business logic in `controller.py`, and persistence in `data_handler.py`, each layer can be updated or replaced independently. This paid off directly in Sprint 3 when the entire view and persistence layers were replaced with React and Firestore. The business rules in `maintenanceService.js` implement the same OK, Due Soon, and Overdue thresholds as `controller.py`, and the rewrite also corrected a known limitation: the Python version approximated months as 30-day periods, while the JavaScript version uses proper calendar-aware arithmetic. MVC made it possible to improve the core logic in isolation without touching the display or data layers.

### Singleton Pattern

- **Where:** `DataHandler` class in `data_handler.py`.
- **How:** Uses a class-level `_instance` and custom `__new__` method to ensure only one instance exists for data management.

**Why Singleton:** If multiple instances of `DataHandler` existed simultaneously, each would hold its own in-memory copy of `vehicles.json`. A write made through one instance would not be visible to the other, leading to silent data loss or vehicles appearing to revert after being updated. Because MotorMinder uses a flat JSON file as its persistence layer rather than a

database with transaction support, the Singleton guarantees that all reads and writes go through exactly one in-memory state, keeping the file and runtime data consistent at all times.

## Command Pattern

- **Where:** `CLI` class in `cli.py`.
- **How:** Maps user menu selections to method calls, acting as a command dispatcher.

**Why Command Pattern:** In `cli.py`, the main menu is implemented as a dictionary that maps user input strings to method references (`actions = {'1': self.list_vehicles, '2': self.add_vehicle, ...}`). The dispatcher then calls whichever method matches the input with no conditional branching. This means adding a new top-level command requires exactly two changes: a new method and a new dictionary entry. No existing dispatch logic is touched, and there is no risk of breaking an existing menu option. The value of this becomes clear when compared to `edit_vehicle()`, which uses a traditional if/elif chain for its sub-menu. That method is noticeably harder to extend because any new edit option requires inserting another elif branch into an already long conditional block.

## Additional Patterns and Principles

**Dataclass/Value Object:** `ServiceRecord`, `Vehicle`, and `ServiceStatus` use Python's `@dataclass` for clean, immutable data structures.

**Adapter (Implicit):** The `Controller` adapts between CLI and model/data layers.

**Enum Usage:** `ServiceName` enum for type-safe service names.

**Composition:** `Controller` composes a `DataHandler` instance.

## Design Trade-Off Discussion

**MVC vs. a single-file script:** A single-file script would have been faster to write for the Sprint 1 prototype but would have made the Sprint 3 migration to React significantly harder. Because business logic was isolated in its own layer, the Sprint 3 rewrite could focus on improving the implementation (such as replacing 30-day month approximations with calendar-aware date arithmetic) without having to untangle display or persistence code at the same time. The main limitation is that MVC adds more files than a small project strictly needs, which increases onboarding time for new contributors unfamiliar with the structure.

**Singleton vs. dependency injection:** Dependency injection would have made the DataHandler easier to swap out in unit tests, which is exactly the problem we encountered in Sprint 2 when the Singleton's cached instance prevented tests from using isolated temp files. We resolved this by resetting `_instance` in each test's setUp, but this is a workaround rather than a clean solution. The Singleton was the right choice for production use given the flat-file persistence model, but a future refactor toward dependency injection would improve testability.

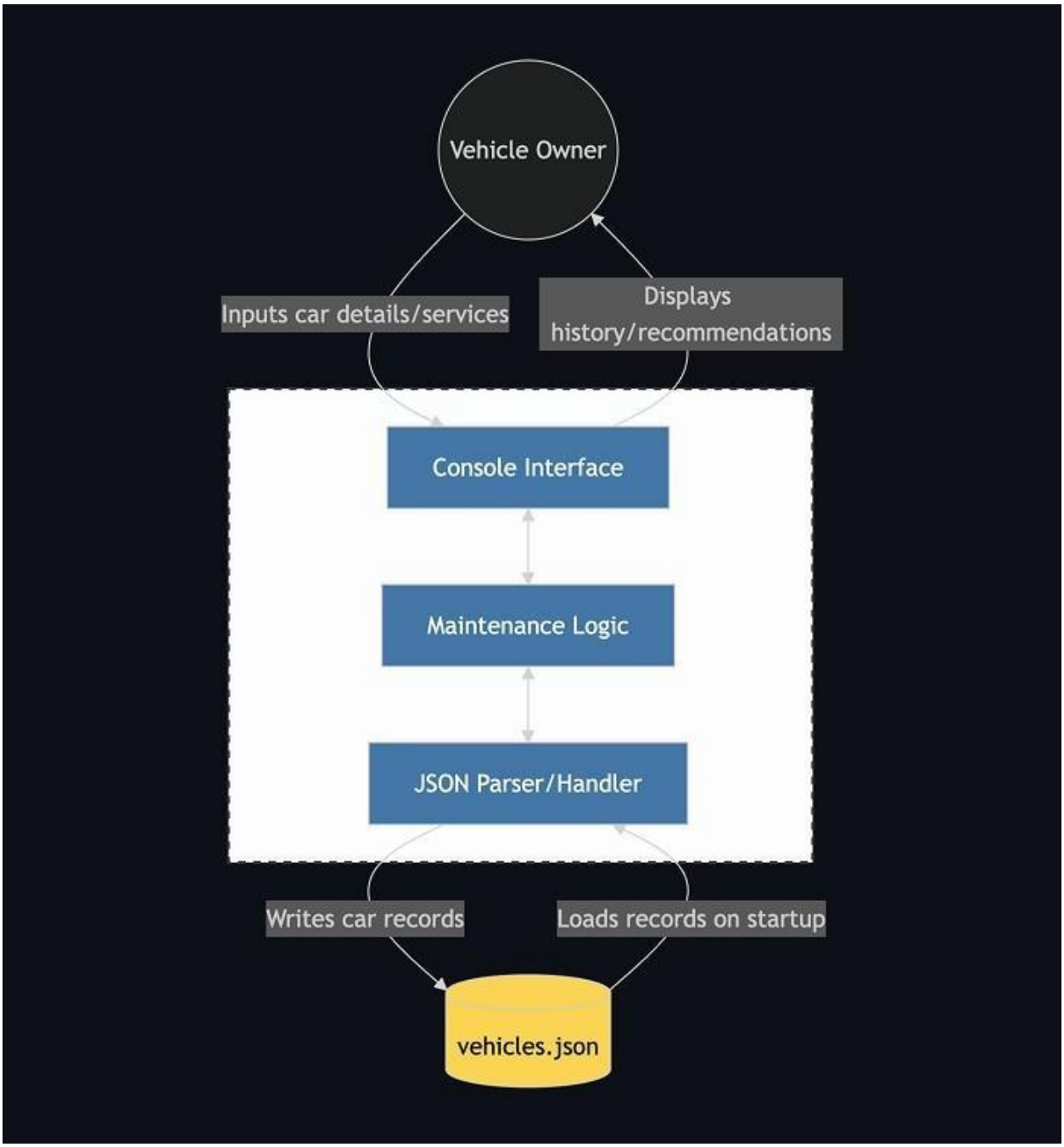
**Command pattern vs. a simple menu loop:** The codebase itself illustrates this trade-off directly. The main menu in `run()` uses the Command pattern via a dictionary dispatcher and is clean and easy to extend. The sub-menu inside `edit_vehicle()` uses a plain if/elif chain and is already harder to extend as a result. The limitation of the Command pattern at this scale is that it adds a small layer of indirection that can feel like over-engineering for a six-option menu. For a project this size, either approach is defensible; the dictionary-based dispatcher earns its complexity by making the main menu consistently extensible.

# Architecture Diagram

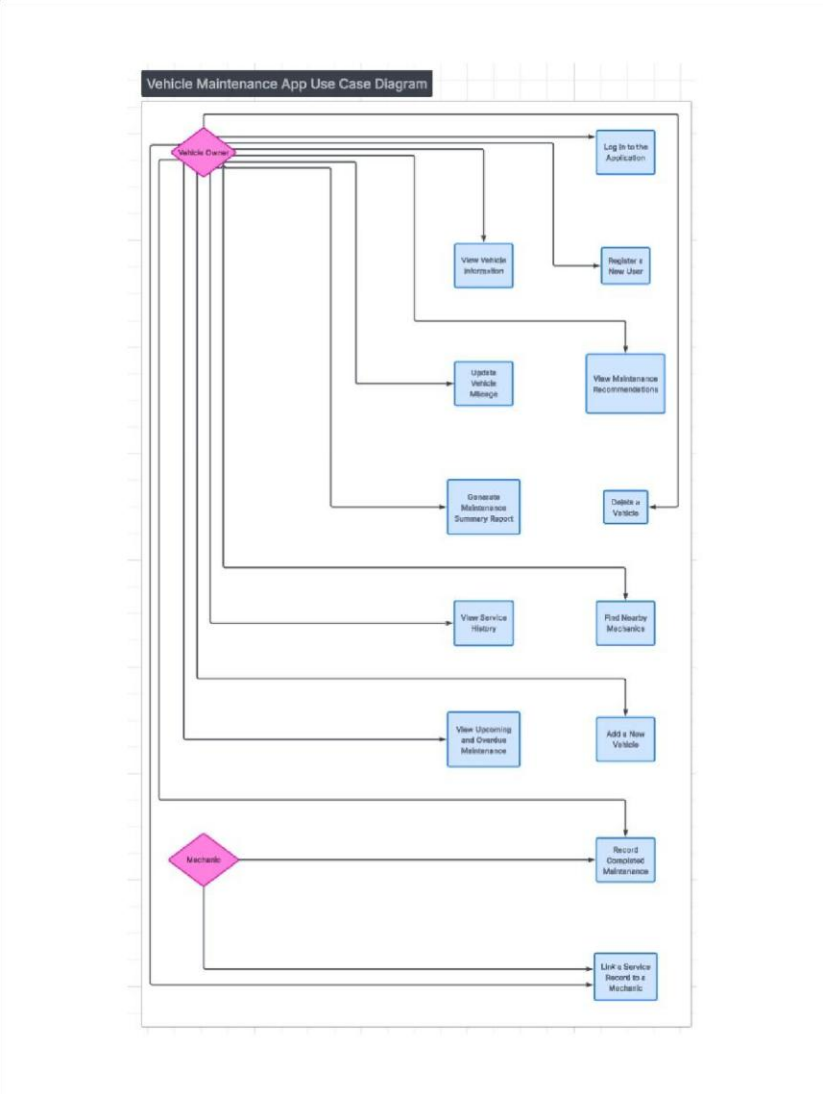
## Level 1: System Context Diagram (MVP)

This diagram follows the C4 Model for software architecture, representing the highest level of abstraction. It illustrates how the MotorMinder application interacts with the user and the local environment.

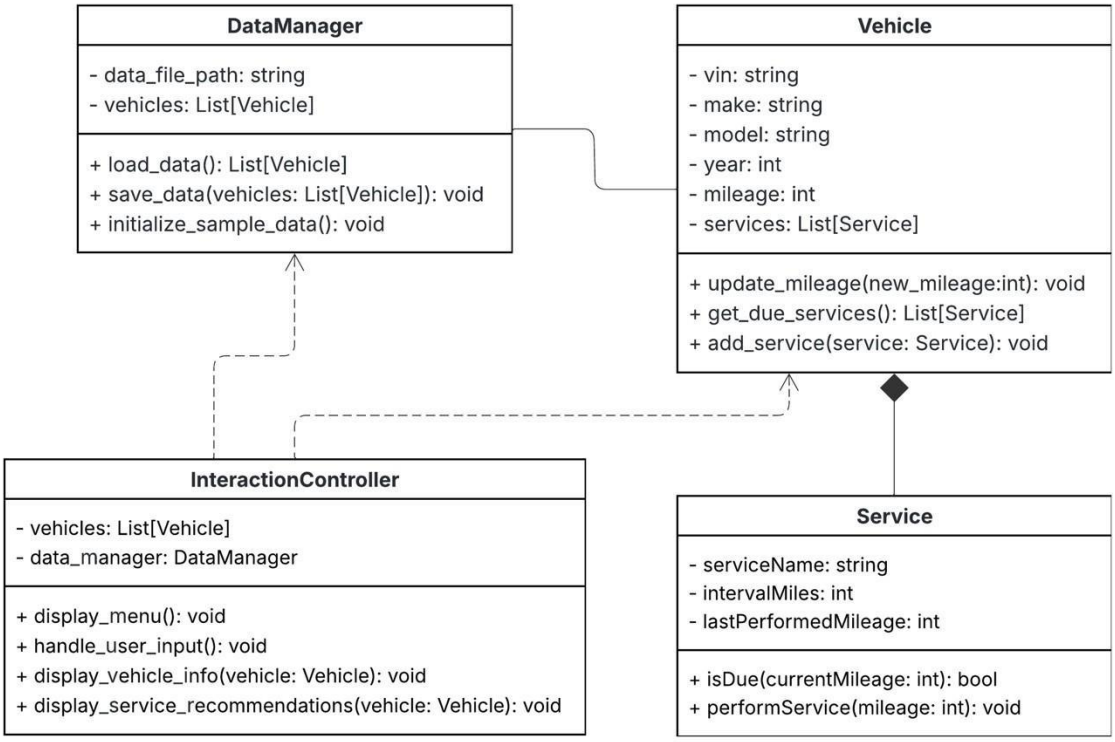
For the Minimum Viable Product (MVP), the system is implemented as a Console-Based Python Application. To ensure simplicity and portability during initial development, the system relies on local JSON persistence rather than external database servers or cloud APIs.


Vehicle Owner
Inputs car details/services
Displays history/recommendations
Console Interface
Maintenance Logic
JSON Parser/Handler
Writes car records
Loads records on startup
vehicles.json

# Use Case Diagram



## Class Diagram




# Testing and QA

## Testing Plan

Sprint 3 testing covers both the legacy Python CLI (MVP) and the new React + Firebase web application. The MVP is tested using pytest with full coverage reporting. The web application is tested using Vitest, targeting the business logic service layer.

### Testing Objectives

- Verify all 41 MVP unit and integration tests pass cleanly after Singleton refactor
- Validate web service logic for maintenance status, mechanic lookup, service intervals, and service logging
- Establish baseline coverage metrics for both codebases
- Identify and document any defects introduced during the Sprint 2 to Sprint 3 migration

## Types of Testing

### *Unit Testing*

Unit tests target individual classes and methods in isolation. The MVP suite covers cli.py, controller.py, data_handler.py, models.py, and view.py. The web suite covers four service modules: maintenanceService.js, mechanicService.js, serviceIntervalService.js, and serviceLogService.js

### *Integration Testing*

Integration tests verify interactions between major components. The MVP suite includes 17 integration tests across four groups: VehicleLifecycleIntegration, ServiceWorkflowIntegration, MechanicLookupIntegration, and CLIControllerIntegration. These tests confirm correct data flow from CLI through controller to persistence layer.

### *System Testing*

System testing is performed manually by running the web application end-to-end in the browser. Testers verify that vehicles can be added, mileage updated, service history logged, and maintenance recommendations displayed correctly under normal usage conditions.

## Test Environment

- MVP: Python 3.10.7, pytest 7.1.3, pytest-cov 7.1.0, Windows 11
- Web: Node.js 22, Vitest 2.1.9, @vitest/coverage-v8 2.1.9, Firebase Emulator
- Data: Temporary JSON files (MVP), Firestore emulator with seed data (Web)
- CI: Manual local execution; automated CI pipeline planned for Sprint 4

## Cross-Sprint QA Metrics Tracking

The following table tracks key QA metrics across all sprints. Sprint 1 did not have an automated test suite. The 14 failures in Sprint 2 were caused by a Singleton refactor that broke the DataHandler and Controller test fixtures; all were resolved in Sprint 3.

| Metric | Sprint 1 | Sprint 2 | Sprint 3 | Sprint 4 |
|---|---|---|---|---|
| **Total Tests** | N/A | 27 passed | 60 passed | TBD |
| **Tests Failed** | N/A | 14 failed | 0 failed | TBD |
| **Pass Rate** | N/A | 66% | 100% | TBD |
| **MVP Coverage** | N/A | N/A | 89% | TBD |
| **Web Service Coverage** | N/A | N/A | 72% (services) | TBD |
| **Bugs Logged** | 1 | 0 | 0 | TBD |

| | | | | |
|---|---|---|---|---|
| **Bugs Resolved** | 0 | 1 | 0 | TBD |

## Trend Analysis

Pass rate improved from 66% in Sprint 2 to 100% in Sprint 3 after resolving the Singleton compatibility issues in the DataHandler and Controller test fixtures. Total test count grew from 27 to 60, reflecting the addition of a full web services test suite. MVP test coverage reached 89% overall, with the main gap in cli.py (64%) due to interactive menu branches that are difficult to unit test without mocking stdin. Web service coverage is 72% on statements; the lower figure reflects that pages, components, repositories, and auth services are not yet covered by automated tests and are validated manually. Sprint 4 targets should include expanding web coverage to the repository layer and adding at least basic component tests.

## Sprint 3 Coverage Breakdown

| Module | Statements | Branches | Functions | Lines |
|---|---|---|---|---|
| cli.py | 64% | — | — | 64% |
| controller.py | 87% | — | — | 87% |
| data_handler.py | 93% | — | — | 93% |
| models.py | 94% | — | — | 94% |
| view.py | 96% | — | — | 96% |
| **MVP Total** | **89%** | **—** | **—** | **89%** |
| maintenanceService.js | 92% | 69% | 100% | 92% |
| mechanicService.js | 100% | 80% | 100% | 100% |
| serviceIntervalService.js | 100% | 100% | 100% | 100% |
| serviceLogService.js | 100% | 100% | 100% | 100% |
| **Web Services Total** | **72%** | **72%** | **91%** | **72%** |

## Bug Report

Defects are documented using a standardized bug report format. Each entry includes a Bug ID, date discovered, full tester name, associated test case, severity, priority, and assigned developer. Severity levels: Critical (system unusable), High (major feature broken), Medium (feature impaired), Low (cosmetic or code quality). Priority levels: High (fix this sprint), Medium (fix next sprint), Low (backlog).

| Bug ID | Date | Tester | Test Case | Severity | Priority | Assigned To |
|---|---|---|---|---|---|---|
| BUG-01 | 01/15/26 | Ethan Hess | Manual Review | Low | Medium | Ryan Carbine |

**BUG-01 Detail:** Missing type annotations, unused imports, and input validation in CLI and models. Identified in Sprint 1 manual review; resolved in Sprint 2. Functions lacked type annotations, unused imports were present in cli.py and models.py, and int(input(...)) calls had no error handling. Environment: Windows 11, Python 3.10.7. Status: Resolved.

## Quality Assessment Metrics

- **Test Coverage:** MVP 89% statement coverage; Web services 72% statement coverage, 91% function coverage
- **Pass/Fail Rate:** Sprint 2: 27/41 (66%); Sprint 3: 60/60 (100%)
- **Defect Density:** 1 defect logged across 1,008 MVP statements = 0.001 defects per statement
- **Mean Time to Fix:** BUG-01 logged Sprint 1, resolved Sprint 2 = 1 sprint
- **Sprint Velocity:** Sprint 1 = MVP prototype; Sprint 2 = CLI feature complete; Sprint 3 = React + Firebase migration with full test suite (60 tests, 89% MVP coverage)

# Customer Feedback

- "There isn't input validation. For example, if I entered '`Year: abc`' the program crashes. Or if I type '`Select vehicle index: 999`' sometimes it handles it and sometimes it doesn't."
- "The mileage logic isn't explained. In the dashboard, I see 'OK', 'Due Soon', and 'Overdue' but I don't know what the threshold is, what 'soon' means, or what rule triggered it."
- "When I edit mileage or make it says updated, but I don't see the new full vehicle details. I'd expect it to reprint the updated vehicle."
- "Logging a service doesn't validate the date. If I enter '`Service data: banana`' it'll likely break when parsed later."